

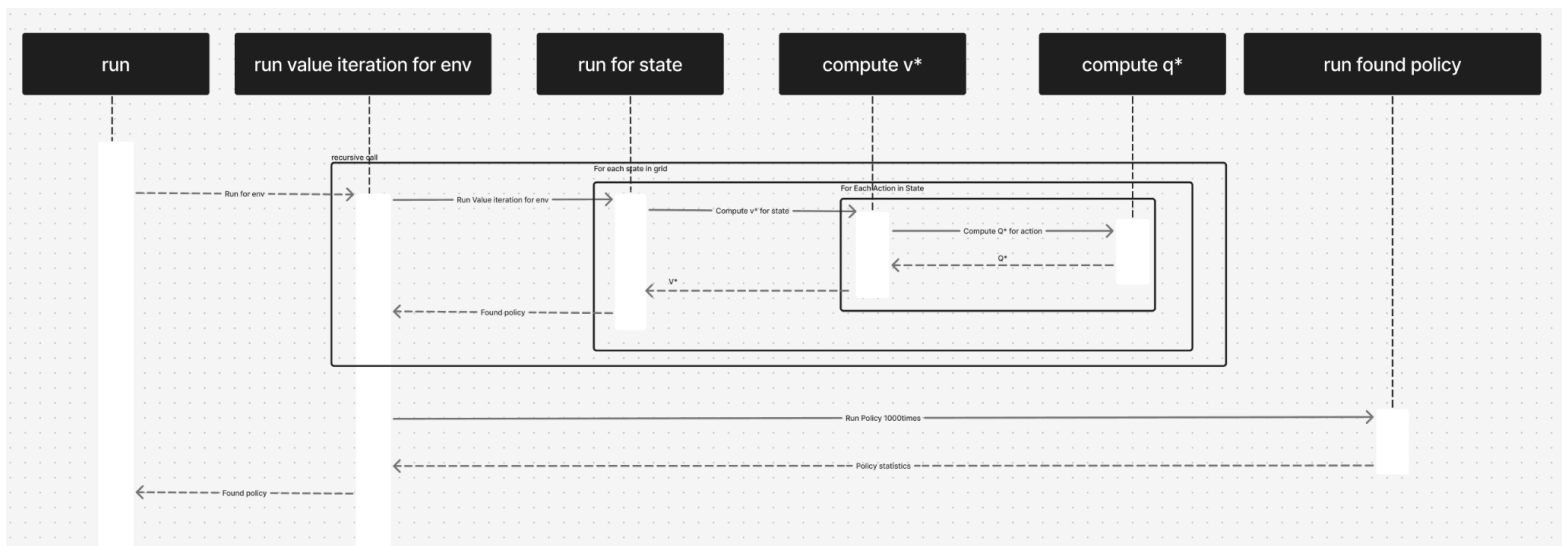
1. Introduction

This assignment consisted of two parts, both evaluating policies, an ordered list of actions, to navigate through an 8×8 grid world. A good policy moves the agent to a terminal goal state with a positive reward (goal), while avoiding terminal states with a negative reward (holes). The first part involved generating and evaluating ten random policies, then identifying the top two among them. The second part required implementing value iteration using the Bellman equation to discover an optimal policy for the same grid. We then compared the performance of this optimal policy against the top-performing policies from part one.

2. Code architecture (0.5 – 1.0 pages max)

Part one is implemented across three methods that run 10 random policies in parallel, every 100 times. It then extracts the top two performing policies, defined as those with the highest average number of successful goal completions over the 100 runs of 10,000 episodes each.

Part two is implemented in a separate file, `value_iteration.py`, using a recursive value iteration algorithm that runs for 1,000 iterations, which is the default recursion stack limit in Python. It aims to discover an optimal policy. Below is a sequence diagram describing the execution path of the algorithm.



<https://www.figma.com/board/YiH5ICJP8HKrrHVQu3yRr5/CSC480-hw3-sequence-diagram?node-id=0-1&t=tuLZU00GYBLm9UnR-1>

3. Computer Specs.

The following results were executed on a 2021 MacBook Pro.

OS	Chip	CPU
MAC OS 26.3.1	Apple M1 Max	64 GB

4. Part 1 Results.

a. Experimental Results:

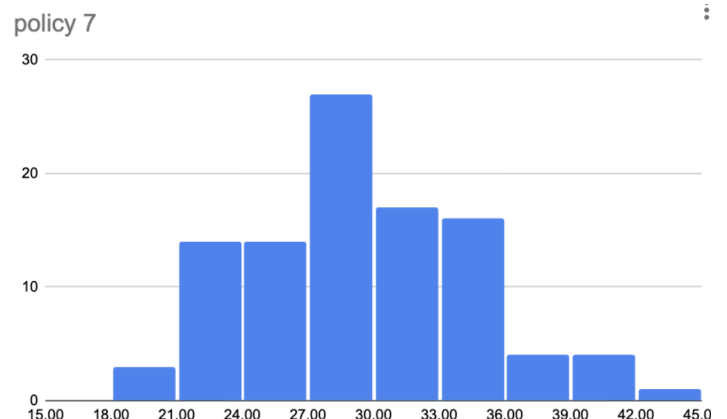
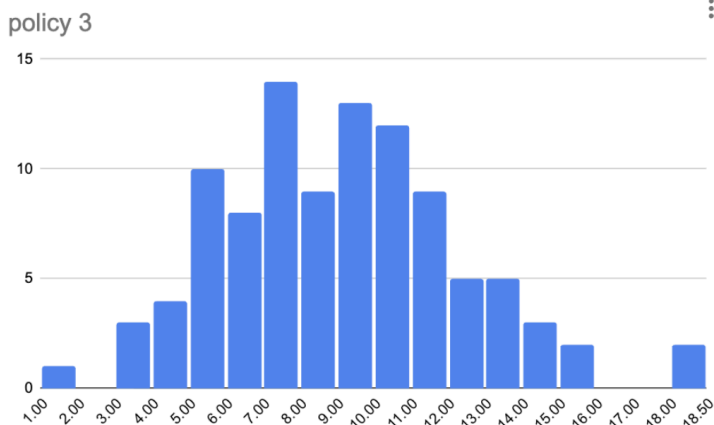
The top two performing policies in this experimental run were policy_7 and policy_3. Below is the analysis of these policies.

	Mean number of episodes, out of the given number, that our player reached the Goal state.	Mean number of steps accumulated over the episodes where the character reached the goal.	Std dev column 1
Policy 7	30.01	2440.52	4.8135
Policy 3	8.98	461.15	3.549

```
*** RESULTS ***:
Name: policy_7
Mean Goals: 30.01
Mean Steps: 2440.52
std dev Goals: 4.813512231209141
[[3 2 2 3 2 3 3 0]
 [0 1 1 3 3 0 1 3]
 [0 3 0 1 3 1 1 1]
 [2 1 3 1 1 2 2 2]
 [2 3 3 3 2 2 1 3]
 [1 0 3 0 3 2 0 0]
 [1 0 0 2 3 1 3 3]
 [3 2 1 2 1 1 1 0]]
```

```
*** RESULTS ***:
Name: policy_3
Mean Goals: 8.98
Mean Steps: 461.15
std dev Goals: 3.549591525795609
[[3 0 0 0 0 3 3 2]
 [0 0 1 1 2 1 1 0]
 [2 2 0 0 1 1 3 2]
 [1 1 2 2 0 2 3 3]
 [3 1 1 2 2 2 3 1]
 [3 0 0 3 3 1 0 1]
 [0 3 2 2 0 1 0 3]
 [1 0 1 2 2 1 1 0]]
```

Density histogram that shows the distribution of the numbers of times theGoal was reached for each policy



b. An in-depth analysis of these results

The two best-performing random policies have success rates of 0.3% and 0.09%. All policies in part one perform poorly. The best seem to contain actions that move the agent in a single direction, which sometimes points toward the goal. Fixed policy can be manually improved over many executions. This process is called policy iteration. The policy is incrementally improved based on its performance. Given the results across all ten random policies, Policy 7 is the strongest candidate for this process.

5. Part 2 Results.

a. Experimental Results:

The value iteration implementation found the following policy

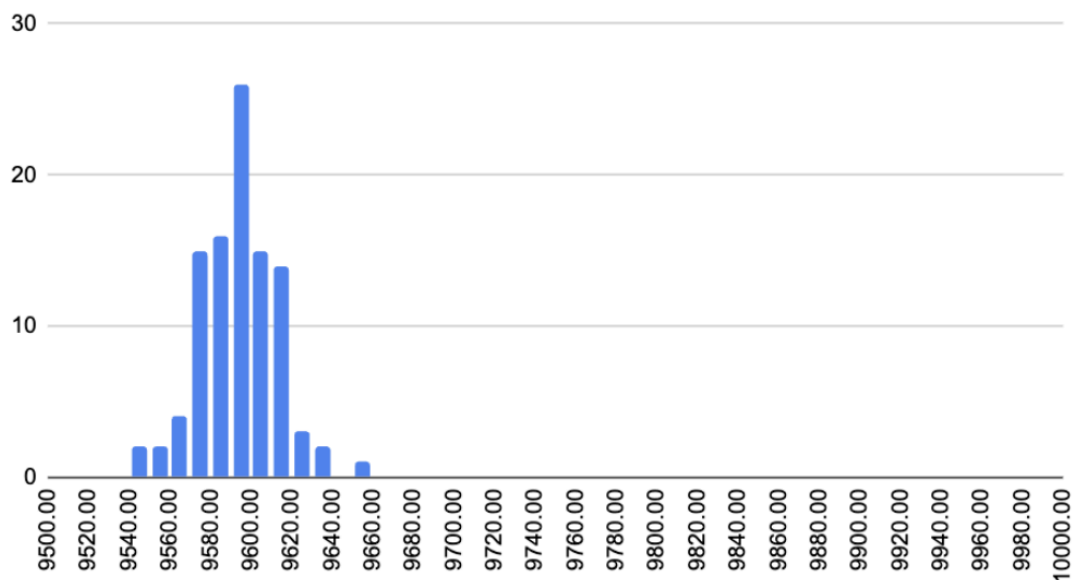
```
optimal policy
[[3 2 2 2 2 2 2 2]
 [3 3 3 3 3 3 2 1]
 [3 3 0 0 2 3 2 2]
 [3 3 0 1 0 0 2 2]
 [0 3 0 0 2 1 3 2]
 [0 0 0 1 3 0 0 2]
 [0 0 2 0 0 0 0 2]
 [0 1 0 0 1 2 1 0]]
```

The table contains statistics gathered for the optimal policy

Mean number of episodes, out of the given number, that our player reached the Goal state.	Mean number of steps accumulated over the episodes where the character reached the goal.	Std dev column 1
9596.99	966246	

Density histogram that shows the distribution of the numbers of times the goal was reached for the optimal policy

Histogram



b. An in-depth analysis of these results

The optimal policy derived from the value iteration implementation significantly outperforms both random policies from Part 1, achieving a 96% success rate compared to 0.3% and 0.09%. The rare failures may be due to the environment's stochastic nature, where the agent is sometimes moved perpendicular to the intended action. Because of this stochastic nature, the recorded policy represents the intended path rather than the executed one, which may not be the same. The policy seems to prioritise avoiding holes over reaching the goal quickly. This is because the algorithm uses a discount rate of 1.0 and a cost-of-living reward of 0, so the algorithm has no motivation to find a shorter path. Changing these values could encourage a shorter path, but also encourage taking risky actions that may lead to a hole.

7. New application (0.25 – 0.5 pages).

The fundamental requirement for applying an MDP is that outcomes depend only on the current state. Many board games and video games satisfy this requirement, where only the current state of the board affects the available moves. MDP could be used to train an agent to play a video game like Overcooked or Cooking Mama. In these games, there is not a binary success or failure terminal state. Instead, players are trying to maximize the number of orders they fill in a given amount of time, this is the agent's utility. By using only the current state, an agent could determine the next best action to maximize its reward. Value iteration can discover the optimal order of actions. These games are also stochastic and timed, food can burn and customers can get angry.

Value iteration can account for these probabilities and be used to discover the optimal policy for a certain level or environment before hand and then execute it at runtime.

8. Feedback

This assignment was necessary to gain an understanding of the value iteration algorithm, and part 1 was helpful for comparing performance. The correct implementation of this algorithm relies on understanding the supporting library, Gymnasium. I found their documentation difficult to understand and reference.

9. Conclusions

From this assignment, I learned the specifics of value iteration and gained an understanding of it by comparing the optimal policy found by the algorithm against random policies. The best of which performed extremely poorly compared to the optimal policy. In the section requiring a new application of MDP, I developed an understanding of the types of problems MDP should be applied to and looked for in the real world around me.